Visual Analytics Using Matplotlib and Seaborn

Visual analytics is the process of exploring, analyzing, and interpreting data using charts, graphs, and visual representations to discover patterns, trends, relationships, and unusual observations

Feature	Matplotlib	Seaborn
Main purpose	General-purpose plotting	Statistical data visualization
DataFrame support	Supported	Excellent built-in support
Customization	Very extensive	Convenient, with Matplotlib customization available
Statistical plots	Possible, but often requires more code	Built-in functions for many statistical plots
Default appearance	Basic	Generally more polished
Best suited for	Fine-grained control and custom charts	Quick exploration of statistical patterns

Frequently used plotting functions

Function	Purpose
plt.plot()	Line chart
plt.bar()	Vertical bar chart
plt.barh()	Horizontal bar chart
plt.scatter()	Scatter plot
plt.hist()	Histogram
plt.pie()	Pie chart
plt.subplots()	Creates a figure and one or more axes
plt.savefig()	Saves a figure to a file
plt.show()	Displays a figure

Line chart using Matplotlib

A line chart is useful for showing how a numerical value changes over an ordered sequence, particularly time

```
sales_df = pd.DataFrame({
    "Month": ["Jan", "Feb", "Mar", "Apr", "May", "Jun"],
    "Sales": [12000, 15000, 13500, 18000, 22000, 25000]
})
plt.figure(figsize=(8, 4))
plt.plot(
    sales_df["Month"],
    sales_df["Sales"],
    marker="o",
    linestyle="-",
    linewidth=2,
    label="Sales"
)
plt.title("Monthly Sales Trend")
plt.xlabel("Month")
plt.ylabel("Sales (₹)")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

plt.figure(figsize=(8, 4)) is used to create a new figure (plotting area) with a specified size

plt.figure() creates a new figure where the graph will be drawn

figsize=(8, 4) specifies the figure's dimensions in inches

8 = width of the figure in inches

4 = height of the figure in inches

Parameter	Explanation
sales_df["Month"]	X-axis values (months)
sales_df["Sales"]	Y-axis values (sales figures)
marker="o"	Displays a circular marker at each data point
linestyle="-"	Connects the data points using a solid line
linewidth=2	Sets the line thickness to 2 points
label="Sales"	Assigns the name "Sales" to the line for use in a legend

label="Sales" assigns the name Sales to the plotted line

plt.legend() displays that name in a legend box on the graph

plt.show() displays the graph

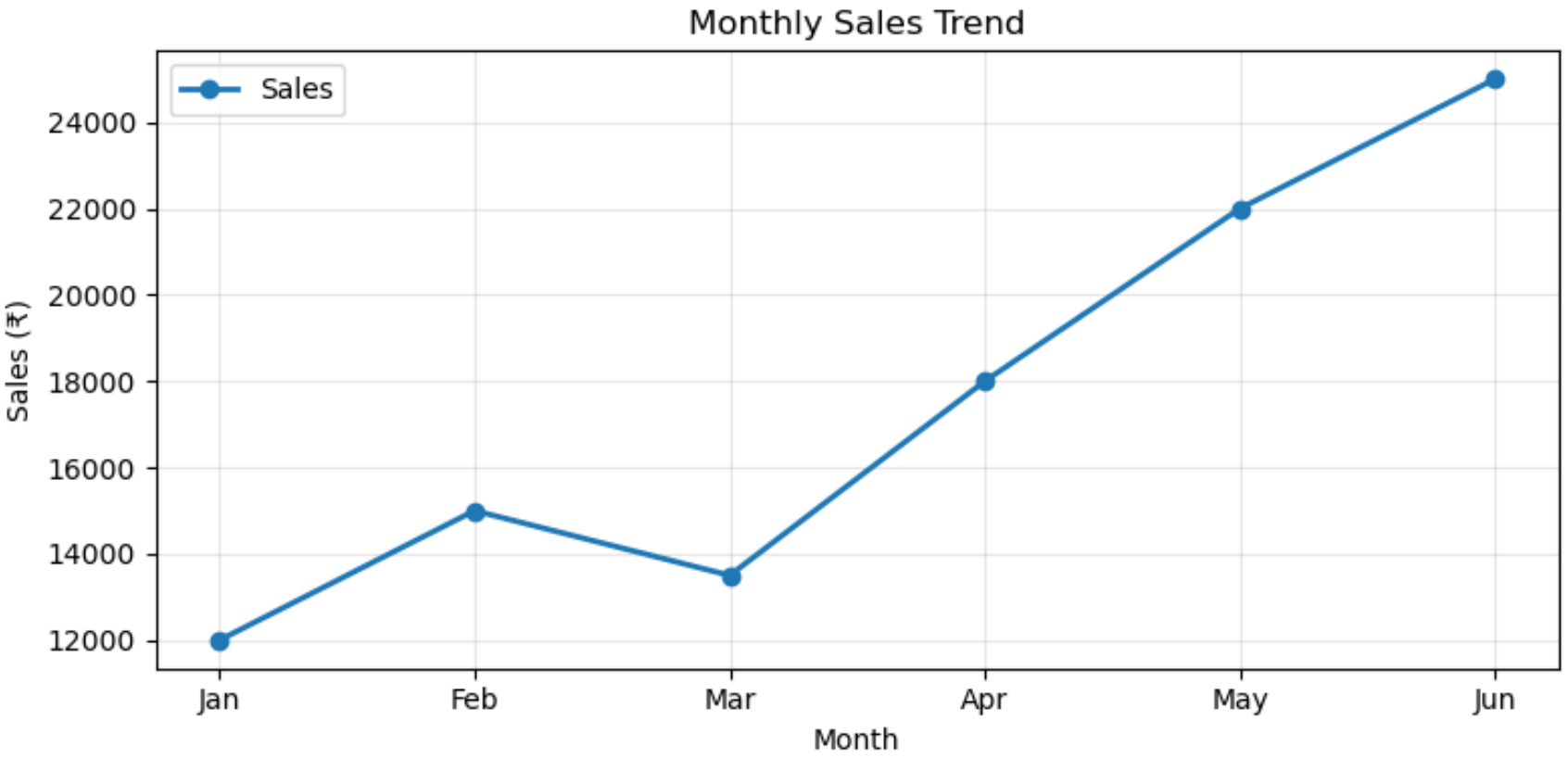
plt.grid() - Controls the display of grid lines

True - Turns the grid lines on

alpha=0.3 Sets the grid-line opacity to 30%, making the lines appear light

plt.tight_layout()

This statement automatically adjusts the spacing around graph elements to prevent labels, titles, and axis names from overlapping or being cut off

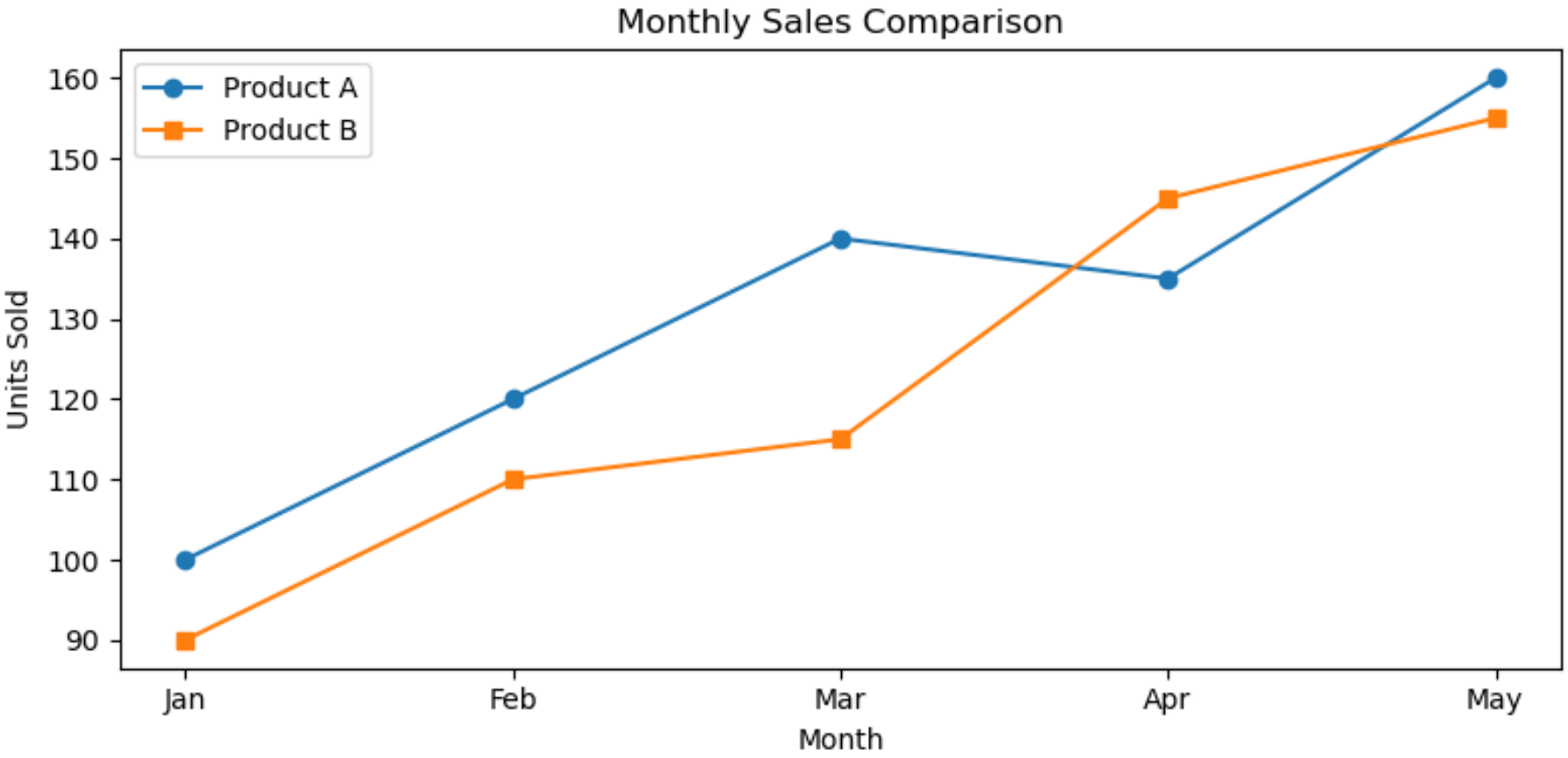

Monthly Sales Trend
Sales
24000
22000
20000
18000
16000
14000
12000
Sales (₹)
Jan
Feb
Mar
Apr
May
Jun
Month

Compare two products

```
sales_df = pd.DataFrame({
    "Month": ["Jan", "Feb", "Mar", "Apr", "May"],
    "Product_A": [100, 120, 140, 135, 160],
    "Product_B": [90, 110, 115, 145, 155]
})
plt.figure(figsize=(8, 4))

plt.plot(sales_df["Month"], sales_df["Product_A"],
         marker="o", label="Product A")
plt.plot(sales_df["Month"], sales_df["Product_B"],
         marker="s", label="Product B")

plt.title("Monthly Sales Comparison")
plt.xlabel("Month")
plt.ylabel("Units Sold")
plt.legend()
plt.tight_layout()
plt.show()
```


Monthly Sales Comparison
Product A
Product B
160
150
140
130
120
110
100
90
Units Sold
Jan
Feb
Mar
Apr
May
Month

```
import matplotlib.pyplot as plt

students = ["Asha", "Rahul", "John", "Priya"]
marks = [65, 80, 45, 90]

plt.plot(students, marks, marker="o")

plt.xlabel("Students")
plt.ylabel("Marks")
plt.title("Student Marks")

plt.ylim(0, 100)

plt.show()
```

Code	Effect
plt.ylim(0, 100)	Y-axis ranges from 0 to 100
plt.ylim(0, 50)	Y-axis ranges from 0 to 50
plt.ylim(20, 100)	Y-axis ranges from 20 to 100
plt.ylim(-10, 100)	Y-axis ranges from −10 to 100

What happens if we change the values?

plt.ylim() changes the visible Y-axis range, not the actual data values

If a data point falls outside the specified limits, it will not be visible within the plotting area

Related function: plt.xlim(0, 10) sets the minimum and maximum values of the X-axis.

Bar chart using Matplotlib

A bar chart compares values across categories

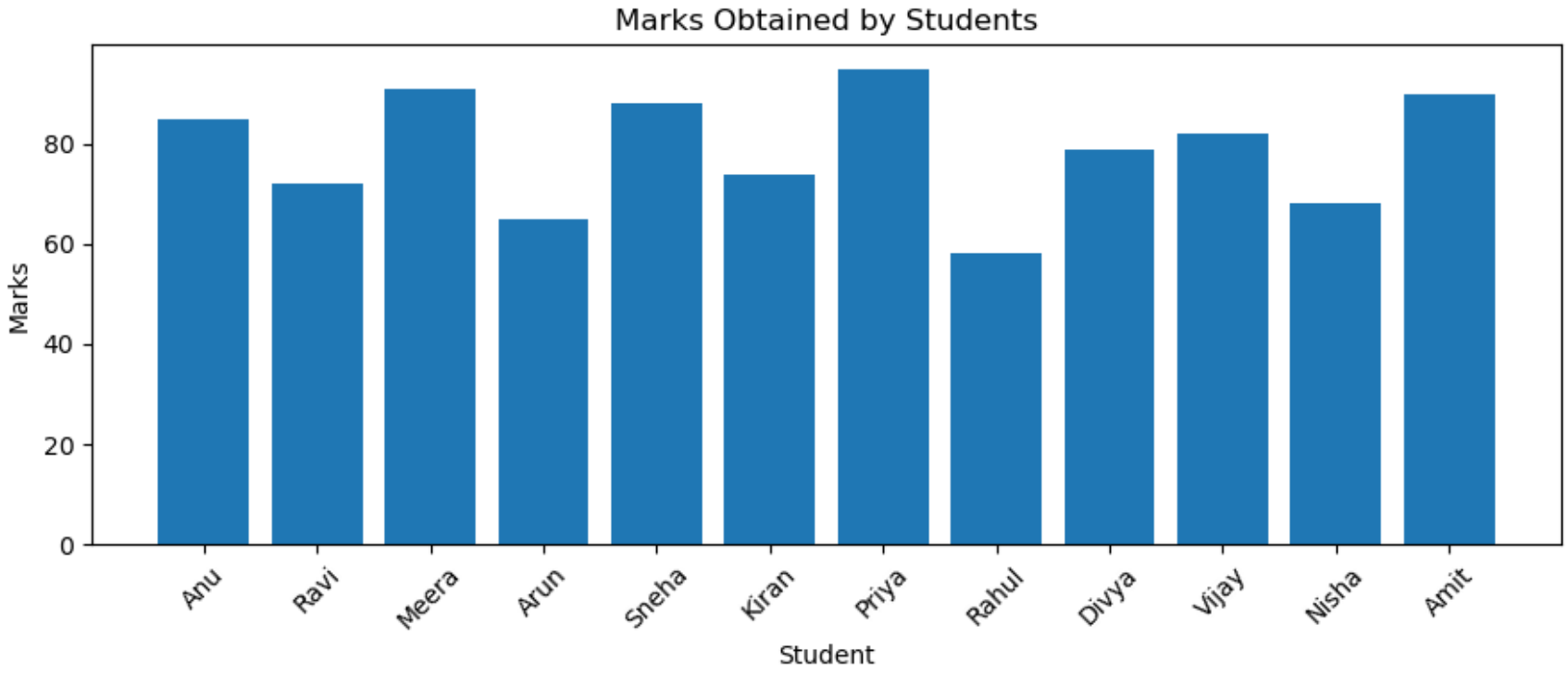
Creating a DataFrame for practical examples

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
data = {
    "Student": [
        "Anu", "Ravi", "Meera", "Arun", "Sneha",
        "Kiran", "Priya", "Rahul", "Divya", "Vijay",
        "Nisha", "Amit"
    ],
    "Department": [
        "CSE", "CSE", "ECE", "ECE", "CSE", "ECE",
        "CSE", "ECE", "CSE", "ECE", "CSE", "ECE"
    ],
    "Marks": [85, 72, 91, 65, 88, 74, 95, 58, 79, 82, 68, 90],
    "Attendance": [92, 78, 95, 68, 90, 80, 98, 60, 85, 88, 72, 94],
    "Study_Hours": [5, 3, 6, 2, 5, 3, 7, 2, 4, 5, 3, 6],
    "Assignment_Score": [90, 75, 94, 68, 89, 77, 96, 60, 82, 85, 70, 92]
}
df = pd.DataFrame(data)
print(df)
```

Inspect the dataset before visualization

```
print(df.head())
print(df.shape)
print(df.columns)
print(df.dtypes)
print(df.describe())
```

```
plt.figure(figsize=(9, 4))
plt.bar(df["Student"], df["Marks"])
plt.title("Marks Obtained by Students")
plt.xlabel("Student")
plt.ylabel("Marks")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```


Marks Obtained by Students
Marks
0
20
40
60
80
Anu
Ravi
Meera
Arun
Sneha
Kiran
Priya
Rahul
Divya
Vijay
Nisha
Amit
Student

plt.xticks() controls the tick marks and labels on the X-axis

rotation=45 rotates the X-axis labels by 45 degrees

This is especially useful when displaying month names, product names, dates, or category labels

when plotting sales for January, February, September, and other long category names, horizontal labels may overlap

Rotating them by 45 degrees can improve readability

Average marks by department

```
dept_avg = df.groupby("Department")["Marks"].mean()

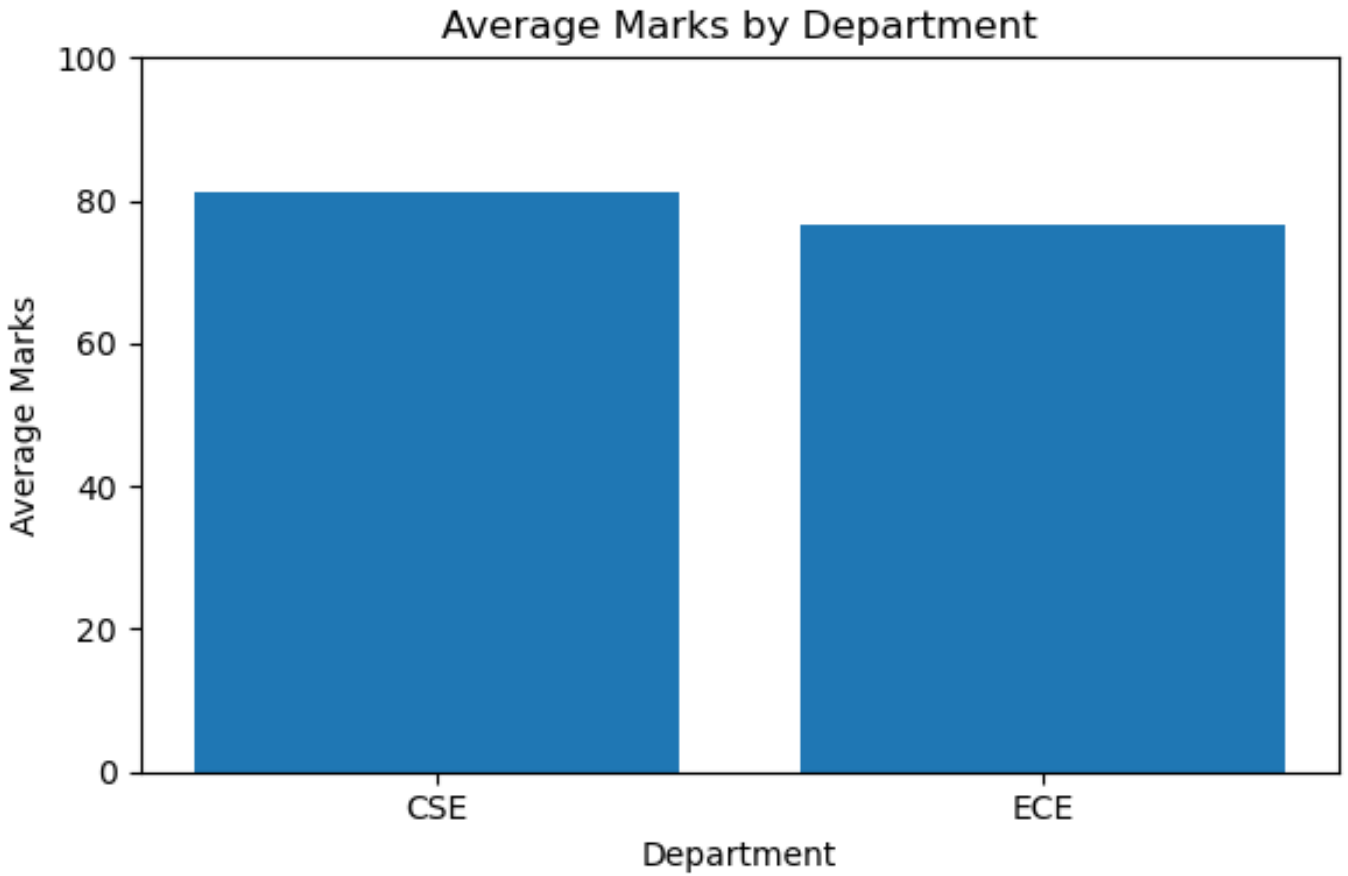
print(dept_avg)

plt.figure(figsize=(6, 4))

plt.bar(dept_avg.index, dept_avg.values)

plt.title("Average Marks by Department")
plt.xlabel("Department")
plt.ylabel("Average Marks")
plt.ylim(0, 100)
plt.tight_layout()
plt.show()
```

plt.ylim() controls the limits of the Y-axis.
0 specifies the minimum Y-axis value.
100 specifies the maximum Y-axis value.
Therefore, the Y-axis displays values from 0 to 100.


Average Marks by Department
100
80
60
40
20
0
Average Marks
CSE
ECE
Department

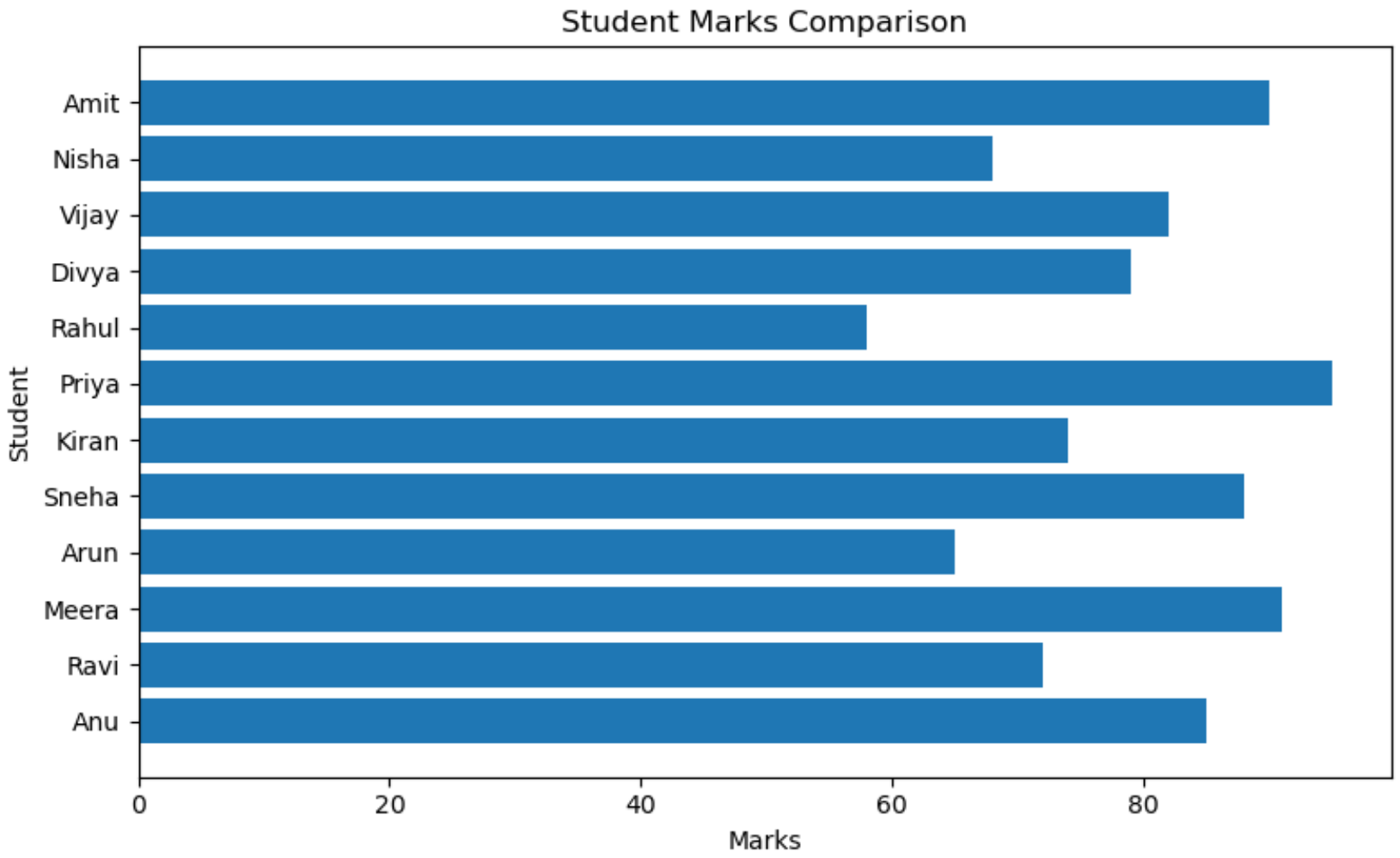
Horizontal bar chart

- Horizontal bars are useful when category labels are long.

```
plt.figure(figsize=(8, 5))

plt.barh(df["Student"], df["Marks"])

plt.title("Student Marks Comparison")
plt.xlabel("Marks")
plt.ylabel("Student")
plt.tight_layout()
plt.show()
```


Student Marks Comparison
Amit
Nisha
Vijay
Divya
Rahul
Priya
Kiran
Sneha
Arun
Meera
Ravi
Anu
Student
0
20
40
60
80
Marks

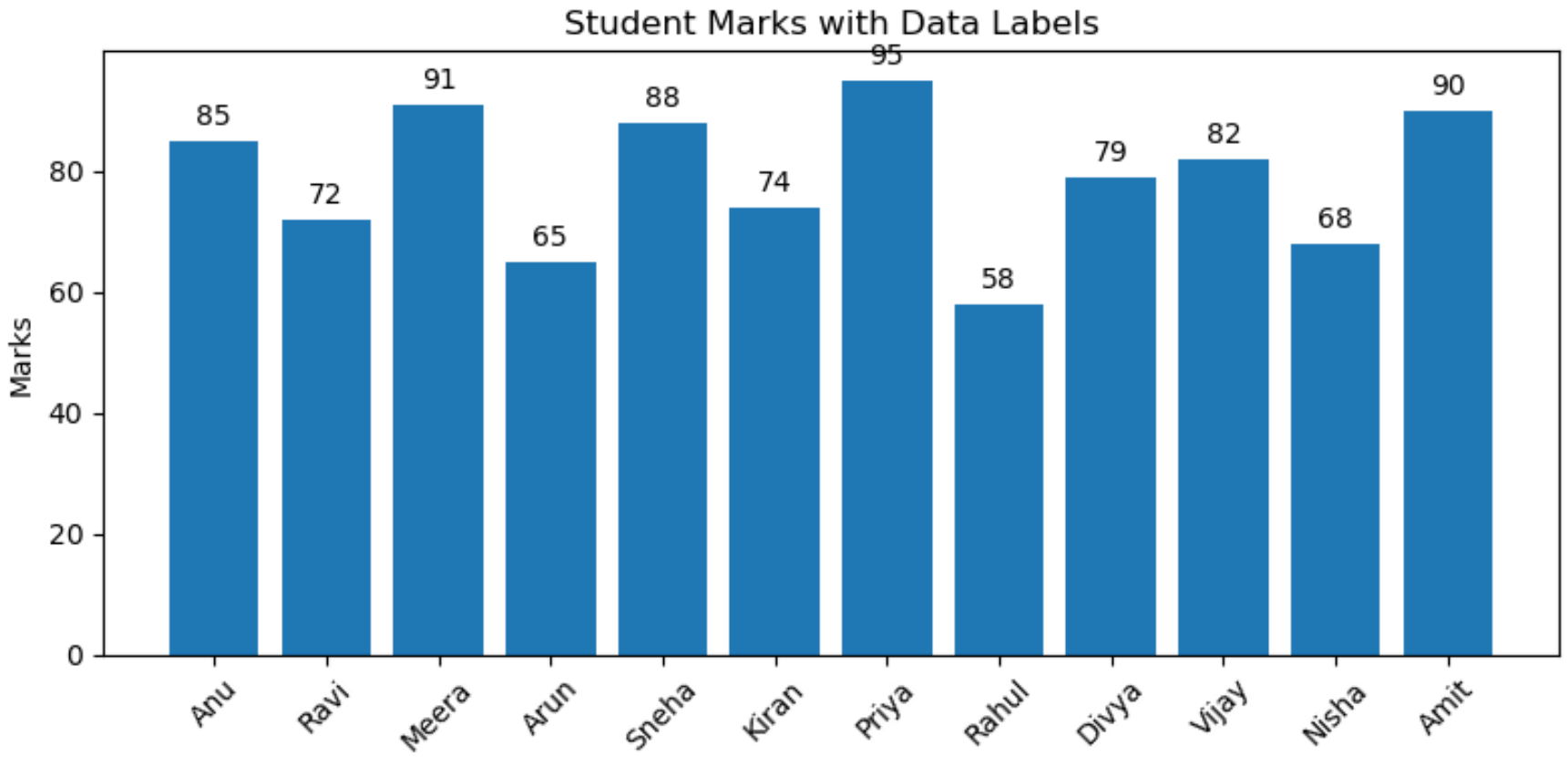
Add values above the bars

```
fig, ax = plt.subplots(figsize=(8, 4))

bars = ax.bar(df["Student"], df["Marks"])

ax.bar_label(bars, fmt="%.0f", padding=3)
ax.set_title("Student Marks with Data Labels")
ax.set_ylabel("Marks")
ax.tick_params(axis="x", rotation=45)

plt.tight_layout()
plt.show()
```


Student Marks with Data Labels
Marks
0
20
40
60
80
85
72
91
65
88
74
95
58
79
82
68
90
Anu
Ravi
Meera
Arun
Sneha
Kiran
Priya
Rahul
Divya
Vijay
Nisha
Amit

fig, ax = plt.subplots(figsize=(8, 4))

This statement creates the figure and the plotting area

plt.subplots() creates a figure and one or more axes

fig refers to the complete figure

ax refers to the axes containing the graph

figsize=(8, 4) sets the figure width to 8 inches and height to 4 inches

```
bars = ax.bar(df["Student"], df["Marks"])
```

This statement creates a vertical bar chart

ax.bar() draws vertical bars

df["Student"] provides the X-axis categories

df["Marks"] provides the height of each bar

bars stores the collection of bar objects returned by Matplotlib.

ax.bar_label(bars, fmt="%.0f", padding=3)

This statement displays the numerical value of each bar above it.

Parameter	Explanation
bars	The bar objects created by ax.bar()
fmt="%.0f"	Displays values as whole numbers without decimal places
padding=3	Adds 3 points of space between each bar's top and its label

For example, the value 75 will appear above Asha's bar

What does fmt="%.0f" mean?

%f formats a number as a floating-point value.

.0 specifies zero digits after the decimal point.

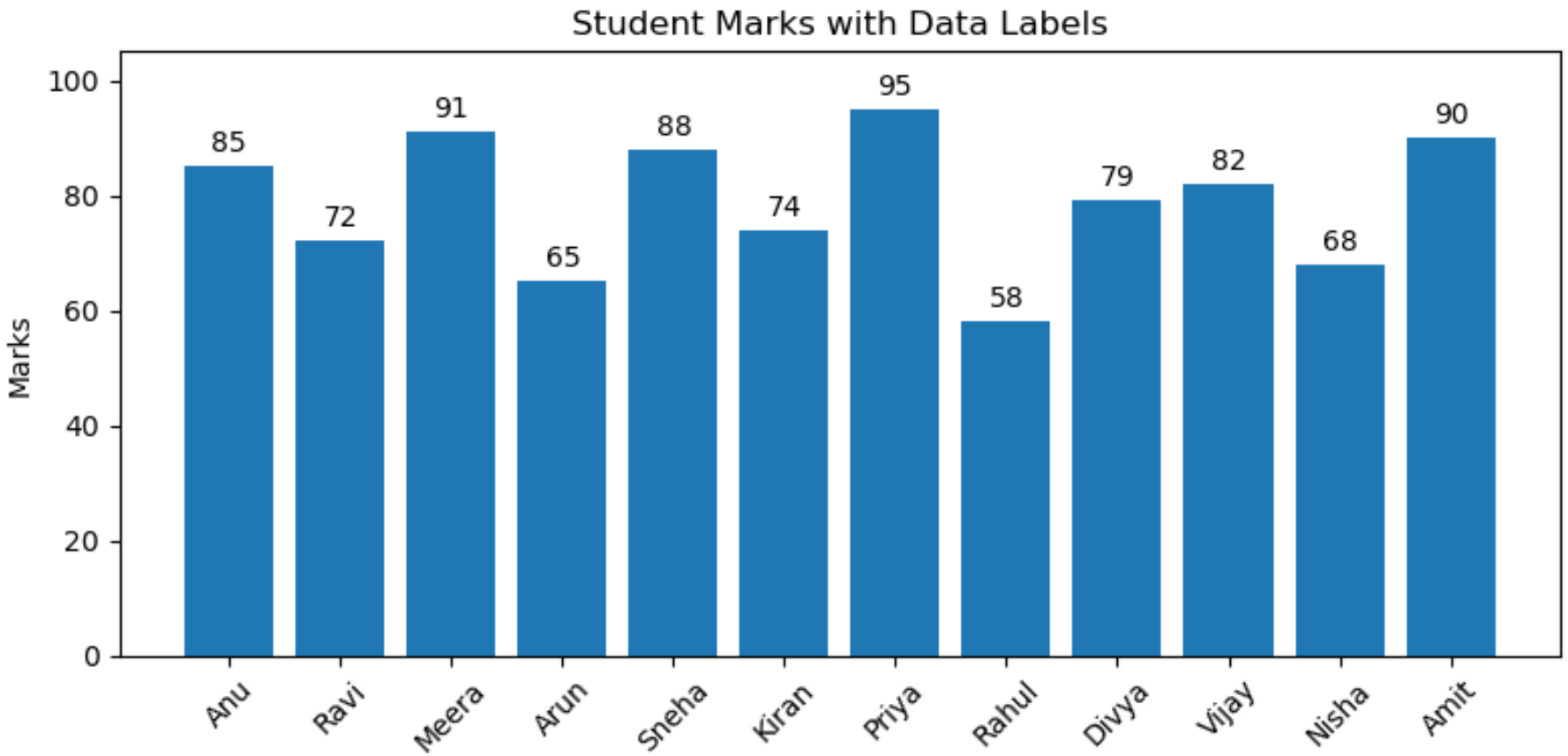
Therefore, 75.0 is displayed as 75.

If you use fmt="%.2f", a value such as 75 will be displayed as 75.00.

- Suppose the highest student's mark is 98, and the data label 98 appears outside the top border

- Use ax.set_ylim() to provide extra space above the tallest bar

```
ax.set_ylim(0, df["Marks"].max() + 10)
plt.tight_layout()
plt.show()
```


Student Marks with Data Labels
Marks
0
20
40
60
80
100
85
72
91
65
88
74
95
58
79
82
68
90
Anu
Ravi
Meera
Arun
Sneha
Kiran
Priya
Rahul
Divya
Vijay
Nisha
Amit

Histogram: Understanding data distribution

- A histogram groups numerical values into intervals called bins and displays how many observations fall within each interval

- Unlike a bar chart, a histogram is primarily used for numerical distributions, not comparisons between unrelated categories

```
plt.figure(figsize=(7, 4))

plt.hist(df["Marks"], bins=6, edgecolor="black")

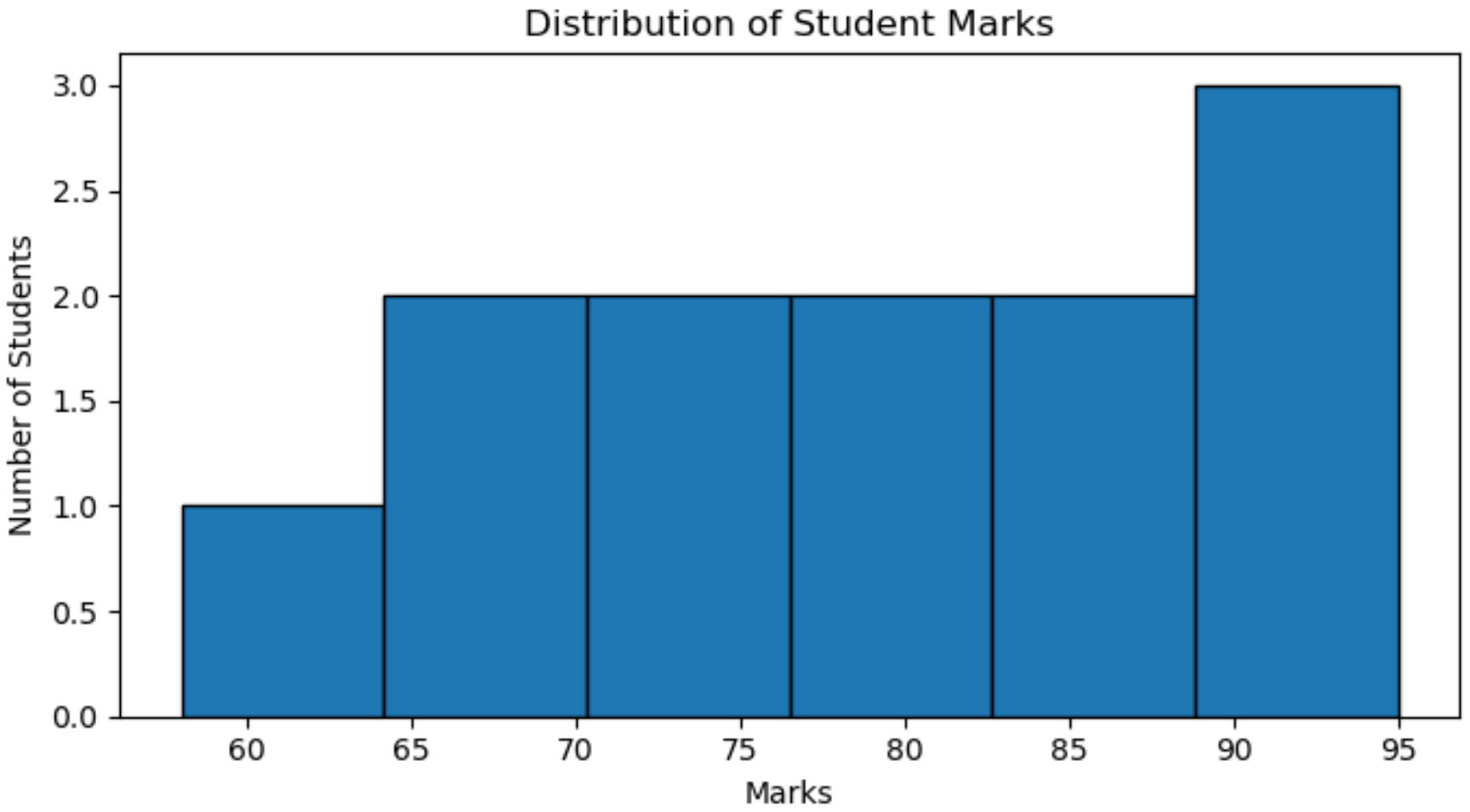
plt.title("Distribution of Student Marks")
plt.xlabel("Marks")
plt.ylabel("Number of Students")
plt.tight_layout()
plt.show()
```

bins=6: divides the numerical range into six intervals

edgecolor="black": outlines the bars

The vertical axis displays the number of observations in each interval

Interpretation: The histogram shows where student marks are concentrated and whether the distribution appears symmetric, skewed, or spread out.


Distribution of Student Marks
Number of Students
Marks
0.0
0.5
1.0
1.5
2.0
2.5
3.0
60
65
70
75
80
85
90
95

Change the number of bins

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

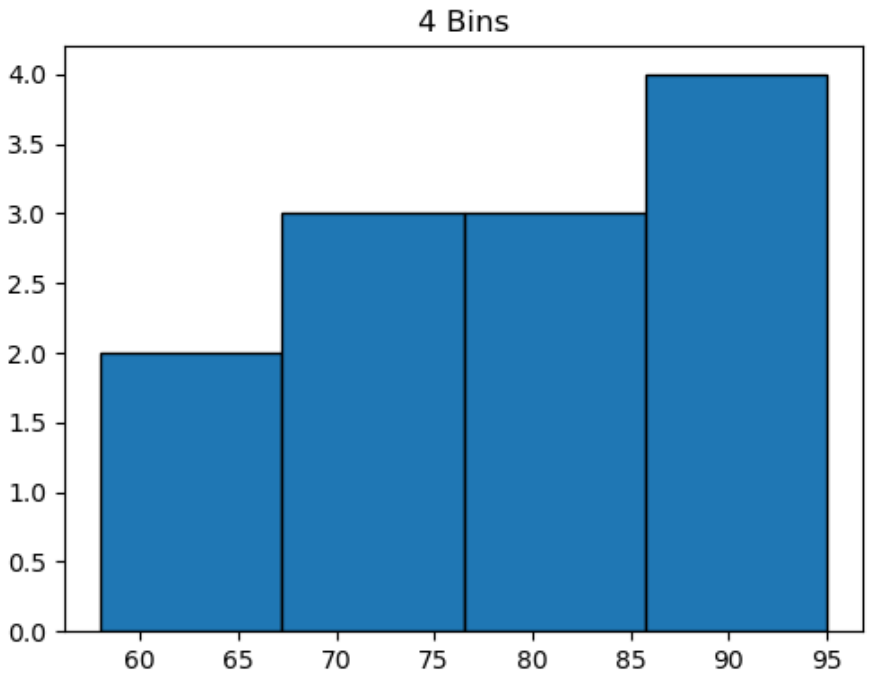
axes[0].hist(df["Marks"], bins=4, edgecolor="black")
axes[0].set_title("4 Bins")

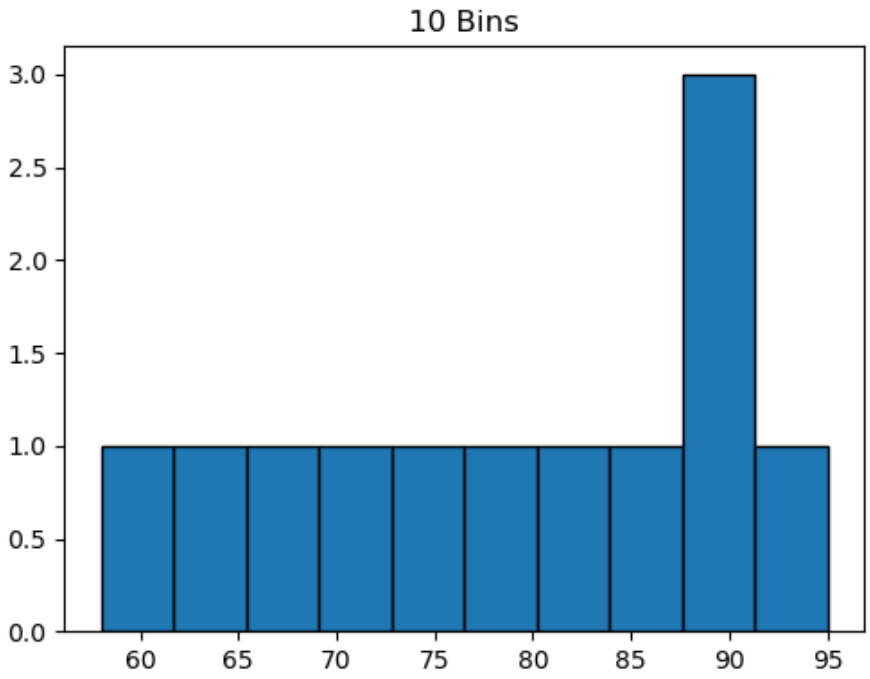
axes[1].hist(df["Marks"], bins=10, edgecolor="black")
axes[1].set_title("10 Bins")

plt.tight_layout()
plt.show()
```

fig, axes = plt.subplots(1, 2, figsize=(10, 4)) creates two graphs side by side within a single figure

Code	Explanation
plt.subplots()	Creates a figure and one or more plotting areas.
1	Specifies 1 row of graphs.
2	Specifies 2 columns of graphs.
figsize=(10, 4)	Sets the total figure width to 10 inches and height to 4 inches.
fig	Stores the complete figure.
axes	Stores the two Axes objects, one for each graph.


4 Bins
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
60
65
70
75
80
85
90
95


10 Bins
3.0
2.5
2.0
1.5
1.0
0.5
0.0
60
65
70
75
80
85
90
95

Thank You..